

Deep Reinforcement Learning (Spring 2025)

Assignment 1

March 2, 2025

In this homework, you will implement methods you learned from the first two weeks to solve the Frozen Lake environment from [Gymnasium](#) (a maintained fork of OpenAI's Gym) under different conditions.

1 About the Environment

Winter is here. You and your friends were tossing around a frisbee at the park when you made a wild throw that left the frisbee out in the middle of the lake.

The water is mostly frozen, but there are a few holes where the ice has melted. If you step into one of those holes, you'll fall into the freezing water. At this time, there's an international frisbee shortage, so it's absolutely imperative that you navigate across the lake and retrieve the disc. However, the ice is slippery, so you won't always move in the direction you intend.

TL;DR: The episode ends when you reach the goal or fall into a hole. You receive a reward of 1 if you reach the goal, and zero otherwise. In the slippery environment, you have a 1/3 possibility of heading in the corresponding direction of your action, and 1/3 of heading each side apart from the opposite direction.

2 Dynamic Programming

In this problem, you will solve the Frozen Lake environment through Dynamic Programming, that is, you are given the dynamics (transition probability) of the environment. Read through `dp.py`. [50pts]

- Implement `policy_evaluation`, `policy_improvement` and `policy_iteration` in `dp.py`. [25 pts]
- Implement `value_iteration` in `dp.py`. [25 pts]
- Run both methods on `FrozenLake-v1` and `SlipperyFrozenLake-v1` environments. In the second environment, the dynamics of the world are stochastic.

3 Temporal Difference Learning

In this problem, you will solve the Frozen Lake environment through Temporal-difference learning. The dynamics of the world are unknown this time. [50 pts]

- Implement `epsilon-greedy-policy` and `sample_action`. [10 pts]
- Implement `Q_learning_step` and `Sarsa_step`. [40 pts]
- Run both methods on the `FrozenLake-v1` environment with `is_slippery=False`.

$$Q^*(s, a) = \mathbb{E}_{s'} \left[R_{t+1} + \max_{a' \in \mathcal{A}} Q^*(s', a') \mid s, a \right]$$
$$Q^*(s, a) \leftarrow R_{t+1} + \max_{a' \in \mathcal{A}} \hat{\mathbb{E}}[Q^*](s', a')$$